Home Broadband Upload Performance — Diagnostic Report

Date: 2026-04-20 **Connection:** 5G Fixed Wireless Access, 20 Mbps upstream / 200 Mbps downstream (advertised) **Equipment:** Outdoor 5G modem → Eero mesh router (customer-owned) **Symptoms:** Sustained uploads collapse to ~5% of line rate and fail partway through large transfers.

---

1. Baseline bandwidth (Apple `networkQuality` — RFC 9097 gold-standard test)

```
Uplink capacity:      20.053 Mbps    Accuracy: High   (13 parallel flows)
Downlink capacity:   188.369 Mbps    Accuracy: High   (16 parallel flows)

Idle Latency:         45.0 ms (HTTP)
Loaded Latency:      665.0 ms (HTTP, under combined up+down load)
Latency penalty:     14.8× idle

Responsiveness:      LOW      174 RPM   (round-trips per minute)
```

Interpretation: The 14.8× jump from 45 ms idle to 665 ms under load, with a “LOW” responsiveness classification, is the textbook signature of bufferbloat. Apple’s `networkQuality` tool specifically flags ISP-side queue-mismanagement when the RTT under load is an order of magnitude worse than idle.

---

2. Real-world sustained upload tests

Two independent destinations, same 100 MB random payload, both HTTPS PUT/POST:

Target	Uploaded	Time (s)	Avg speed	Result
`us-ashburn-1.ocir.io` (Oracle)	70.3 / 100 MB	600.0	117 KB/s (0.94 Mbps)	⏱ TIMED OUT at 70%
`httpbin.org` (AWS Virginia)	45.2 / 100 MB	439.8	103 KB/s (0.82 Mbps)	❌ HTTP 502 after 45% (peer RST)
Docker push of 380 MB image (real)	~280 MB	~540	~0.8 Mbps	❌ 3× connection reset by peer at different blobs

Key observation: Actual sustained upload throughput is ~1 Mbps, roughly 5 % of the 20 Mbps advertised uplink measured by `networkQuality`'s own short-duration test. The discrepancy between burst-capacity (20 Mbps) and sustained-throughput (1 Mbps) is the clearest quantitative fingerprint of:

- **Policed / shaped sustained uploads** (ISP rate-limits flows above some duration threshold), OR
- **Buffer-induced TCP collapse** (huge upstream queue → ACKs delayed so far that TCP's RTO fires → spurious retransmits → flow stalls → eventually the CPE or ISP drops the session).

Both uploads ended in connection resets, not graceful completions.

---

3. Latency stability

Pings to various hops (10 packets each, 0.2 s interval):

Target	Loss	Min	Avg	Max	Stddev
Home CPE (`192.168.4.1`)	0 %	3.6	39.9	197.9	**57.6**
ISP first hop (`10.48.39.69`)	0 %	3.8	45.5	123.5	**41.7**
Cloudflare `1.1.1.1` (internet control)	0 %	19.5	33.8	47.8	**9.1**

The home-CPE ping itself — staying inside my own LAN — shows 57 ms stddev and a 197 ms max. A healthy router should answer pings in <5 ms with near-zero variance. This means the queueing is happening very close to the edge (CPE/modem or the ISP's first-hop aggregation), not on the wider internet path. Cloudflare `1.1.1.1` by contrast shows clean 9 ms stddev.

Ping during upload vs idle:

```
Idle pings to 1.1.1.1 (post-upload):   22 – 58 ms   stddev 13.7 ms
During-upload pings to 1.1.1.1:        14 – 219 ms  stddev 55.4 ms   (~4×
worse)
```

RTT explodes the moment a sustained upload starts — because packets destined outbound are queueing in oversized buffers somewhere on the upstream path.

---

4. Path characterisation (traceroute)

```
 1  gateway (in-home mesh, <1ms)
 2  10.48.39.69              12 ms       (CPE edge — first ISP hop)
 3  10.49.250.250           403 ms       ← ISP aggregation hop, wildly
elevated
 4  172.24.40.234            343 ms
 5  10.160.164.174           367 ms
 6  172.24.195.5             268 ms
 7  172.24.195.6             321 ms
 8  Zayo (transit)            70 ms
 9  Zayo LHR28 UK             36 ms
11  Zayo LHR11 UK            133 ms
12  Lumen LON1                36 ms
13  Lumen WASHINGTON12       111 ms
14  Lumen (4.16.72.246)      120 ms
15  Oracle edge (140.204.220.182) 98 ms
```

Hops 3–7 inside the ISP network show 268–403 ms RTTs — on a path that ultimately reaches a Zayo hand-off at 70 ms. Hops beyond the ISP consistently show lower RTTs than hops inside the ISP. That's a textbook indication that queueing/congestion is occurring within the ISP's own network, not on the public internet.

---

5. What's been ruled out

Hypothesis	Evidence against
Oracle OCIR is broken	AWS `httpbin.org` fails identically
My Mac's TCP stack	`netstat -s` shows 0 retransmit timeouts at idle
DNS	OCIR small requests resolve + succeed in <1 s
Packet loss	0 % loss to `1.1.1.1`, CPE, and ISP hop at idle
IPv6 misconfig	Happy-eyeballs falls back to IPv4 cleanly
MTU / PMTUD	Small uploads and downloads (188 Mbps) work fine
Firewall	HTTPS handshakes to both targets complete OK

Every failure mode correlates with **sustained outbound throughput**. Everything idle and everything short-burst works cleanly.

---

6. What this means for the ISP

Two specific questions to raise with the provider:

1. **Is there an upload traffic-policing or shaping rule that throttles sustained flows after a few MB?** The 20 Mbps burst → 1 Mbps sustained gap is consistent with aggressive policing.

2. **Is AQM / `fq_codel` enabled on the upstream path** (modem/CMTS or DSLAM / OLT)? The 14.8× idle→loaded latency jump and the hop-3 RTT of 400 ms inside the ISP network both point to undersized / unmanaged buffers.

If the modem is ISP-supplied, the first thing to check is whether the modem-level shaping profile is correct for the subscribed plan. In many cases an outdated shaping profile is the root cause.

---

7. Reproduction recipe (for the ISP engineer)

```
# Baseline
/usr/bin/networkQuality -v                    # Apple's RFC-9097 tester

# Sustained upload to a neutral HTTPS endpoint
dd if=/dev/urandom of=/tmp/blob.bin bs=1m count=100
curl -w "%{http_code} %{speed_upload}\n" -X POST --data-binary @/tmp/blob.bin
        \
     https://httpbin.org/anything --max-time 600

# Ping-under-load (in a 2nd terminal during the upload)
ping -c 30 -i 0.5 1.1.1.1
```

Expected on a healthy 20 Mbps uplink: ~45 s to completion, speed ~2 MB/s, ping stddev <30 ms. Observed here: never completes, ~100 KB/s, ping stddev 55 ms.

---

8. Impact on real work

- Docker image pushes to OCIR (Oracle Container Registry) die after 1–3 blob uploads and require 5–10 retries to eventually complete a deploy.
- Git pushes of even modest size (5–20 MB) frequently stall.
- Any kind of `rsync` / `scp` to cloud bastions needs `--partial --inplace` restarting the transfer repeatedly.
- All observed symptoms resolve immediately when using a mobile hotspot (same MacBook, identical commands).

Net impact: ~2 hours/day of engineering time lost to retries and workarounds.